

Detection Strategy

Pre-processing, Engineering, and Classification

Before statistical tests are applied, the solution structures the data to make sure the detection is robust.

1. **Dynamic Feature Classification:** Ignores column names and uses programmatic heuristics to bucket features into numerical, binary, cat_low (low cardinality), cat_high (high cardinality).
2. **Missingness Indicators:** Automatically detects features whose missing value rates shift by more than 5% between train and test sets.
3. **Rank Transformation:** Converts numeric features to percentile ranks to provide resistance to global shifts in magnitude.

Data Drift Detection

Classified features are routed to specific statistical tests for drift detection.

1. **Numerical Features (KS-Test & PSI):** $O(N)$ approximate Kolmogorov-Smirnov (KS) test alongside the Population Stability Index (PSI).
 - a. Zero-Inflated Guardrails: Recognizing telecommunications data (e.g., roaming charges, out-of-bundle data) often has massive clusters of zeros, the pipeline splits continuous data into "zero mass" and "non-zero mass" distributions. It calculates a zero_mass_diff to detect if customers simply stopped using a service, distinct from shifts in the usage amounts of active users.
2. **Categorical Features (Cramer's V & JSD):** *
 - a. Low-Cardinality: Uses Chi-Squared and Cramer's V to detect market-share shifts in small-menu options.
 - b. High-Cardinality: Uses Jensen-Shannon Divergence (JSD) for sparse, high-dimensional features where traditional Chi-Squared tests fail mathematically.
3. **Time-Aware Trend Detection:** It looks at monotonic median trends over time to identify features that are steadily degrading or inflating, allowing the pipeline to anticipate future degradation

Adaptive Mitigation Strategy

The solution's mitigation is a dynamic 'self-healing' system. Its interventions are scaled based on mathematical severity of the previously detected drift.

1. **Automated & Stability-Gated Feature Pruning:** When features are deemed too toxic or volatile, it is removed to protect the model from false patterns.
 - a. **Adversarial Dropping (Separability Pruning):** The `apply_adversarial_dropping` function acts as a purely mathematical filter. It calculates the Kolmogorov-Smirnov (KS) statistic for numeric features and the Total Variation (TV) distance for categorical features between the train and test sets. If a feature's distribution is highly separable (exceeding adaptive thresholds), it is automatically dropped. This prevents the model from exploiting "alien" features that only exist in one domain.
 - b. **Stability-Gated Pruning:** Located in `apply_stability_gated_pruning`, this acts as a conservative pruning pass specifically for high-cardinality features. It calculates a composite "risk score" that multiplies drift severity by temporal instability (how much the feature's relationship with the target fluctuates over time). Features exceeding the risk threshold are dropped.
 - c. **Structural Mitigation:** The `mitigate_structure` function handles foundational data corruption by silently dropping corrupted primary keys (like Excel E+ scientific notation corruption) and aggressively binning rare labels in high-cardinality columns into an `other_rare` category to prevent overfitting.
2. **Adaptive Distribution Alignment & Winsorization:** Instead of discarding drifted numeric features, the solution attempts to "re-align" them so the model can still extract predictive value.
 - a. **Adaptive Distribution Alignment (Quantile Binning):** Combates magnitude inflation (e.g., global data usage doubling over a year). If a numeric feature fails the KS test, the pipeline uses `pd.qcut` to convert the absolute raw values into 10 equal-sized percentiles (0-9). This forces the model to learn relative rank (e.g., "this user is in the top 10% of users") rather than absolute values, completely neutralizing scalar drift.
 - b. **Calibrated Winsorization:** Cap extreme outliers for drifted numericals using Interquartile Range (IQR) bounds. For features experiencing exponential range explosion, it dynamically applies log scaling. Furthermore, it calibrates the train median to the test median (`test_median_ref`), actively shifting the test data distribution closer to the training domain before inference.
3. **Dynamic & Fallback Target Encoding (TE):** High-cardinality categorical variables are notoriously fragile to drift. The pipeline uses a dynamic encoding strategy to stabilize them.

- a. **Mathematically Scaled Smoothing:** Dynamically adjusts the smoothing penalty applied to categories. If a feature shows high Total Variation (TV) distance between train and test sets, the smoothing penalty is drastically increased. This pulls the encoded value closer to the global average, reducing the model's reliance on drifted categories.
 - b. **Automated Fallback Strategies:** Actively downgrades how it handles features based on drift severity:
 - i. **Moderate Drift:** Falls back to simple Frequency Encoding.
 - ii. **Severe Drift:** Triggers "Neutralization," where the categories are completely wiped and replaced with the global target mean.
 - c. **Leakage-Safe K-Fold OOF:** To prevent overfitting on the training data, `apply_kfold_target_encoding_with_plan` uses an Out-Of-Fold (OOF) encoding mechanism built on Polars window expressions, blending it with globally learned maps for test-time inference.
- 4. **Mathematical Covariate Shift Weighting:** To address covariate shift, the solution re-weights the training data.
 - a. **Temporal Similarity Weighting:** Functions like `get_temporal_similarity_weights_fast` evaluate how mathematically similar each historical training month is to the unseen test set using KS distances. Training months that look statistically identical to the test environment receive higher sample weights, while outdated or anomalous months are down-weighted. This achieves temporal adaptation without human scheduling.
 - b. **Spatial Distance Weighting (1-NN):** The `get_distance_weights` function builds a cKDTree across normalized numeric features. For every test row, it mathematically locates the nearest training row. Training rows that are frequently identified as "neighbors" to test rows get a gentle weight boost. This achieves the exact same goal as traditional Adversarial Validation weighting, but strictly avoids training an illicit secondary AI classifier.
- 5. **Explicit Shift Encoding:** explicitly feeds the context of the drift into the model.
 - a. **Categorical Prior Shift Ratios:** The `add_categorical_shift_ratio_features` function calculates $\log(p_{\text{test}} / p_{\text{train}})$ for categories. If a specific subscription plan suddenly becomes 5x more common in the test set, this dynamic feature explicitly quantifies that shift ratio. This allows the LightGBM model to adjust its internal decision trees based on the popularity trajectory of the category, rather than just its static value.

References

Data Drift & Covariate Shift Research

Sugiyama, M., Krauledat, M., & Müller, K. R. (2007). Covariate Shift Adaptation by Importance Weighting. *Journal of Machine Learning Research*, 8, 985-1005. Retrieved from <https://www.jmlr.org/papers/v8/sugiyama07a.html>

Huang, J., Smola, A., Gretton, A., Borgwardt, K., & Schölkopf, B. (2006). Correcting Sample Selection Bias by Unlabeled Data. *Advances in Neural Information Processing Systems (NeurIPS)*. Retrieved from <https://proceedings.neurips.cc/paper/2006/hash/a2186aa7c086b46ad4e8bf81e2a3a19b-Abstract.html>

Lipton, Z., Wang, Y. X., & Smola, A. (2016). Detecting and Correcting for Label Shift with Black Box Predictors. *arXiv preprint*. Retrieved from <https://arxiv.org/abs/1607.01719>

Nowozin, S. (n.d.). Effective Sample Size in Importance Sampling. *Sebastian Nowozin Blog*. Retrieved from <https://www.nowozin.net/sebastian/blog/effective-sample-size-in-importance-sampling.html>

NannyML. (n.d.). Types of Data Shift. *NannyML Blog*. Retrieved from <https://www.nannyml.com/blog/types-of-data-shift>

Evidently AI. (n.d.). Data Drift Detection for Large Datasets. *Evidently AI Blog*. Retrieved from <https://www.evidentlyai.com/blog/data-drift-detection-large-datasets>

Wikipedia Contributors. (n.d.). Importance Sampling. *Wikipedia, The Free Encyclopedia*. Retrieved from https://en.wikipedia.org/wiki/Importance_sampling

Wikipedia Contributors. (n.d.). Domain Adaptation. *Wikipedia, The Free Encyclopedia*. Retrieved from https://en.wikipedia.org/wiki/Domain_adaptation

Encoding, Transforms & Validation

scikit-learn developers. (n.d.). `sklearn.preprocessing.QuantileTransformer`. *scikit-learn documentation*. Retrieved from <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.QuantileTransformer.html>

scikit-learn developers. (n.d.). `sklearn.model_selection.TimeSeriesSplit`. *scikit-learn documentation*. Retrieved from https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html

scikit-learn developers. (n.d.). Cross-validation: evaluating estimator performance. scikit-learn documentation. Retrieved from https://scikit-learn.org/stable/modules/cross_validation.html

scikit-learn-contrib. (n.d.). TargetEncoder. Category Encoders documentation. Retrieved from https://contrib.scikit-learn.org/category_encoders/targetencoder.html

Train In Data. (n.d.). MeanEncoder. Feature-engine documentation. Retrieved from https://www.feature-engine.trainindata.com/en/latest/user_guide/encoding/MeanEncoder.html

Practical Drift Tooling & Repositories

Evidently AI. (n.d.). Evidently GitHub Repository. GitHub. Retrieved from <https://github.com/evidentlyai/evidently>

Evidently AI. (n.d.). Evidently README. GitHub. Retrieved from <https://raw.githubusercontent.com/evidentlyai/evidently/main/README.md>

NannyML. (n.d.). NannyML GitHub Repository. GitHub. Retrieved from <https://github.com/NannyML/nannyml>

NannyML. (n.d.). NannyML README. GitHub. Retrieved from <https://raw.githubusercontent.com/NannyML/nannyml/main/README.md>

SeldonIO. (n.d.). Alibi-Detect README. GitHub. Retrieved from <https://raw.githubusercontent.com/SeldonIO/alibi-detect/master/README.md>